CSE 151A - Homework 7

due Thursday, May 22 at 11:59PM

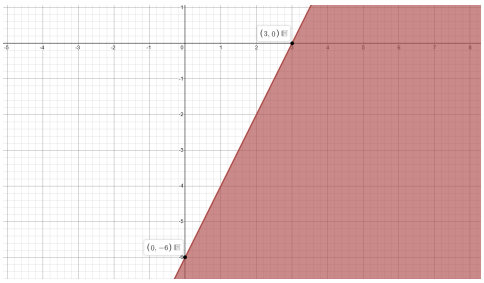
Instructions:

- You may discuss problems with your study group, but ultimately all your work (mathematical problems, code, experimental details) must be individual.
- Your solutions must be `typed up` and uploaded to Gradescope by 11.59PM on Thursday May X. No late homeworks will be accepted under any circumstances, so you are encouraged to upload early.
- A subset of the problems will be graded.

Problem 1.

Draw the decision boundary in $\mathbb{R}^2$ that corresponds to the prediction rule $\text{sign}(2x_1 - x_2 - 6)$. Make sure to clearly indicate where this boundary intersects the axes. Show which side of the boundary is classified as positive and which side as negative.

Solution: The shaded region is the set of points that are classified as positive, and the unshaded region is the set of points that are classified as negative.



Problem 2.

A particular labeled data of n points is randomly permuted and then the Perceptron algorithm is run on it, repeatedly cycling through the points until convergence. It converges after making k updates. For each of the following statements, say whether it is **definitely true** or **possibly false**, and give a brief reason.

a) The data is linearly separable.

> **Solution:** This is definitely true because the Perceptron algorithm converges if and only if the data is linearly separable.

b) If the process were repeated with a different random permutation, it would again converge.

> **Solution:** This is definitely true because convergence is guaranteed regardless of permutation since the data is linearly separable.

c) If the process were repeated with a different random permutation, it would again converge after making k updates.

> **Solution:** This is possibly false because the number of updates can vary with different permutations due to the order in which mistakes are corrected.

d) k is at most n.

> **Solution:** This is possibly false because the number of updates can exceed n as the algorithm may cycle through the data multiple times.

Problem 3.

The Perceptron algorithm is run on a data set, and converges after performing $p + q$ updates. Of these updates, p are on data points whose label is -1 and q are on data points whose label is $+1$. What is the final value of the parameter b?

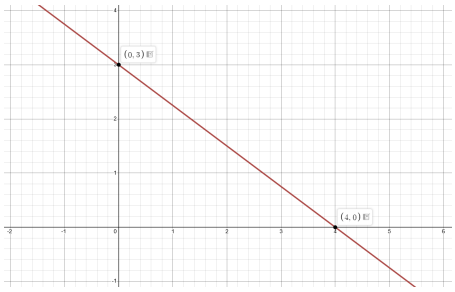
> **Solution:**
> $$b = \sum_{i=1}^{q} 1 + \sum_{i=1}^{p} -1 = q - p$$

Problem 4.

An SVM classifier is learned for a data set in $\mathbb{R}^2$. It is given by $w = \begin{bmatrix} 3 \\ 4 \end{bmatrix}$ and $b = -12$.

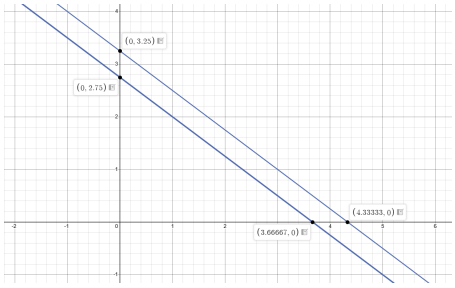
a) Draw the decision boundary, making sure to clearly indicate where it intersects the axes.

Solution:



b) Draw the left-hand and right-hand boundaries, also clearly marking where they intersect the axes.

Solution:



c) What is the margin of this classifier?

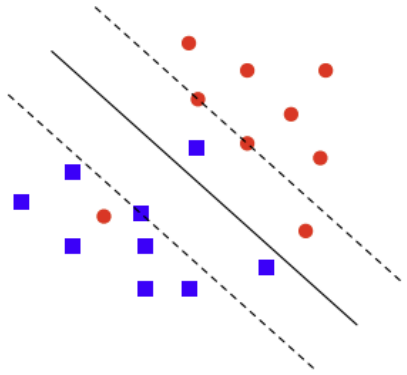
Solution: The margin is the distance between the two boundaries, which is $\frac{2}{\|w\|} = \frac{2}{\sqrt{3^2+4^2}} = \frac{2}{5}$.

d) How would the point $\begin{bmatrix} 2 \\ 2 \end{bmatrix}$ be classified?

Solution: The point $\begin{bmatrix} 2 \\ 2 \end{bmatrix}$ is classified as positive because $w \cdot x + b = 3(2) + 4(2) - 12 = 2 > 0$.

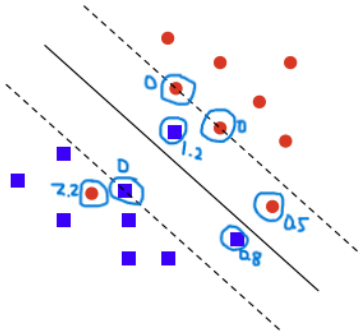
Problem 5.

The picture below shows the decision boundary obtained upon running soft-margin SVM on a small data set of blue squares and red circles.



a) Copy this figure and mark the support vectors. For each, indicate the approximate value of the corresponding slack variable.

Solution:



b) Suppose the factor C in the soft-margin SVM optimization problem were increased. Would you expect the margin to increase or decrease?

Solution: Increasing C places a higher penalty on slack variables (misclassifications or margin violations). This forces the SVM to reduce violations by fitting the data more strictly, which usually results in a smaller margin to better separate the points.

Problem 6.

(This problem is based on material from discussion section.) The dual form of the Perceptron algorithm is used to learn a binary classifier, based on n training points. It converges after k updates, and returns a vector α and a number b. For each of the following statements, indicate whether it is necessarily true or possibly false, and give a brief justification.

a) Each α_i is either 0 or 1.

Solution: This is possibly false because each α_i could be updated multiple times, so it is possible for it to be greater than 1.

b) $\sum_{i=1}^{n} \alpha_i = k$.

Solution: This is definitely true because the sum of the α_i is equal to the number of updates k.

c) α has at most k nonzero coordinates.

Solution: This is definitely true because there were k updates, so there are at most k coordinates that were updated and are non-zero.

d) The training data must be linearly separable.

Solution: This is possibly false because the dual form of the Perceptron algorithm can converge even if the data is not linearly separable.

Programming Problem 7. *Binary Perceptron*

In this problem, you will code up the (binary) Perceptron algorithm and use it to classify the Iris data set.

a) Write code for two functions:

- The first function takes as input parameters w, b of a linear classifier as well as a data point x, and returns the label for that point: $\text{sign}(w \cdot x + b)$. The label is either $+1$ or -1.
- The second function takes as input an array of data points and an array of labels (where each label is $+1$ or -1), and runs the Perceptron algorithm to learn a linear classifier w, b. The algorithm should begin by randomly permuting the data points.

In your writeup, give the code for these two functions.

```
def classify(x, w, b):
    return np.sign(np.dot(x, w) + b)

def perceptron(X, Y, epochs=100):
    perm = np.random.permutation(len(X))
    X = X[perm]
    Y = Y[perm]

    w = np.zeros(X.shape[1])
    b = 0
    updates_per_epoch = []

    for _ in range(epochs):
        updates = 0
        for x, y in zip(X, Y):
            if classify(x, w, b) != y:
                w += y * x
                b += y
                updates += 1
        updates_per_epoch.append(updates)
        if updates == 0:
            break

    return w, b, updates_per_epoch
```

b) Load in the Iris data set. You can do this by simply invoking:

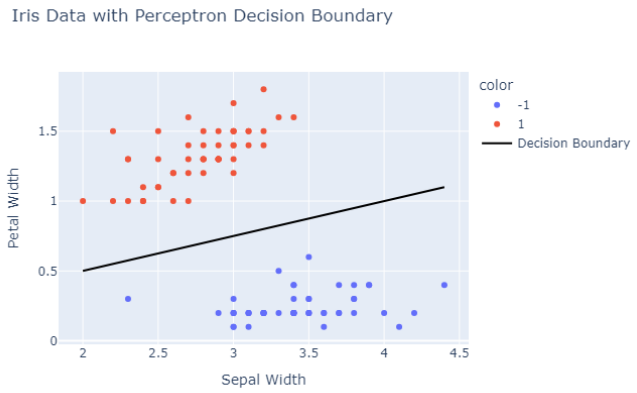
```
from sklearn import datasets
iris = datasets.load_iris()
X = iris.data
Y = iris.target
```

The data has four features and three labels. Restrict it to features 1 and 3 (the second and fourth columns, sepal width and petal width) and to labels 0,1. Recode label 0 as -1, since this is what the Perceptron algorithm is expecting.

```
X = X[Y != 2]
X = X[:, [1, 3]]
Y = Y[Y != 2]
Y[Y == 0] = -1
```

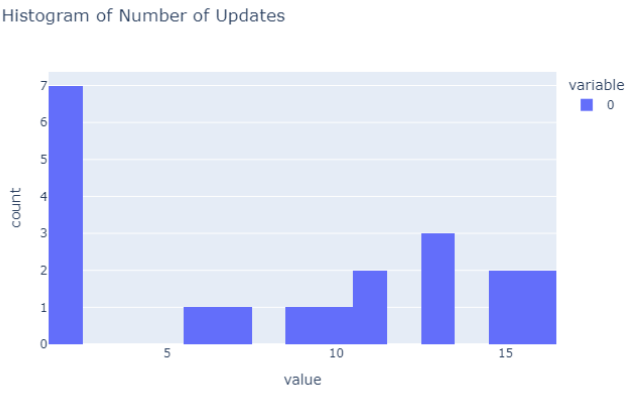
c) Now run the Perceptron algorithm on the data. In your writeup, show a plot with the data points (where the two labels have different colors) and the resulting decision boundary.

Solution:



d) Now modify your code from part (a) to count the *number of updates* made by the Perceptron algorithm while it is learning. Run the algorithm 20 times and keep track of the number of updates needed each time. In your writeup, include a histogram of these values.

Solution:



Programming Problem 8. *Support vector machine*

As you did with the Perceptron, use the Iris data set, but this time use features 0 and 2, and labels 1,2.

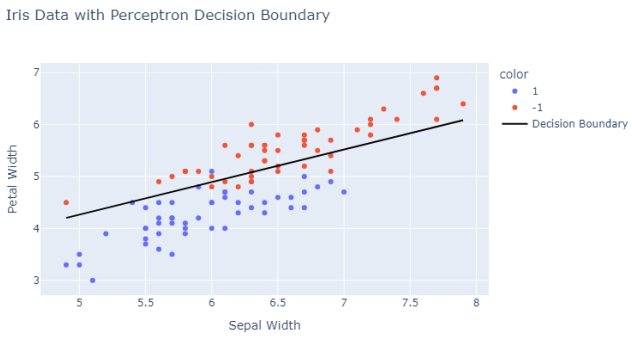
a) Is this data linearly separable?

> **Solution:** No, the data is not linearly separable because the convex hulls of the two classes overlap.

b) Use `sklearn.svm.SVC` to fit a support vector machine classifier to the data. You will need to invoke the option `kernel='linear'`. Try at least 10 different values of the slack parameter C. In your writeup, include a table that shows these values of C and for each of them gives the training error and the number of support vectors.

> **Solution:**
>
> | C | Training Error | Support Vectors |
> |---|---|---|
> | 0.0001 | 0.17 | 100 |
> | 0.001 | 0.17 | 100 |
> | 0.01 | 0.16 | 92 |
> | 0.1 | 0.07 | 56 |
> | 1 | 0.07 | 32 |
> | 10 | 0.05 | 18 |
> | 100 | 0.05 | 14 |
> | 1000 | 0.05 | 14 |
> | 10000 | 0.05 | 14 |
> | 100000 | 0.06 | 14 |

c) Which value of C do you think is best? For this value, include a plot of the data points and the linear decision boundary.

> **Solution:** I would choose $C = 10$ because it's the lowest C with the lowest training error. Choosing a higher C would increase the chance of overfitting.
>
> 
>